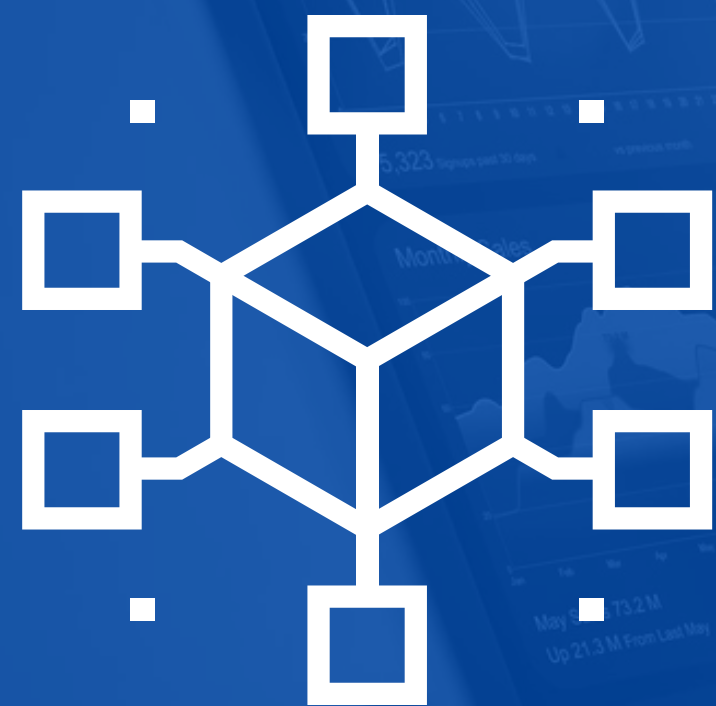




PROJECT DEMO

Global Wardrobe DWH

By: Beka Chokuri

BUSINESS BACKGROUND

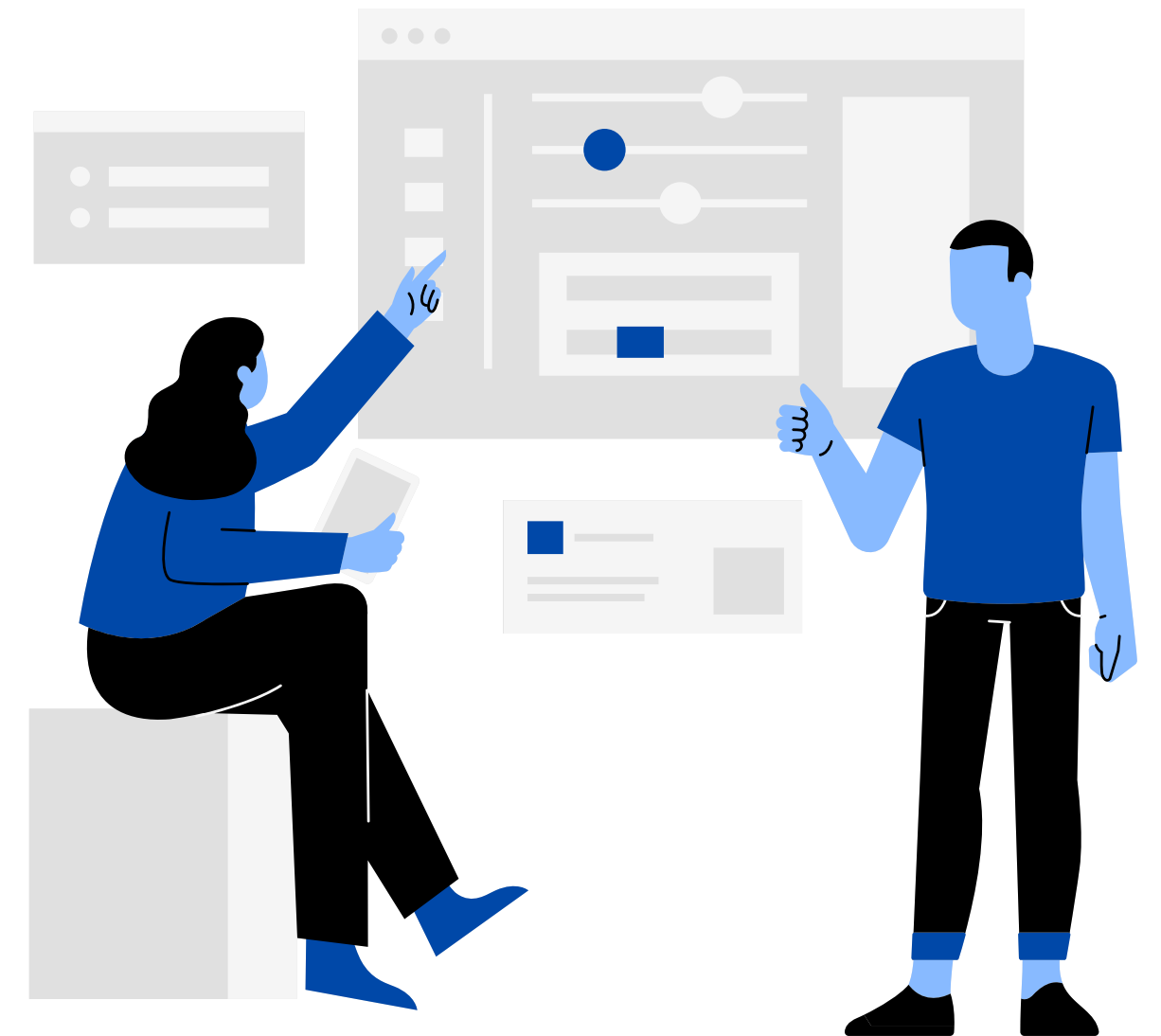
"Global Wardrobe" represents a dynamic retail enterprise specializing in clothing and accessories. The company strategically operates across the United States' domestic market and international territories, encompassing European and Asian regions. Its diverse product portfolio spans clothing lines, decorative ornaments, and complementary accessories, designed to satisfy varied consumer preferences and emerging fashion trends. Through analysis of sales data from multiple markets, the organization aims to optimize inventory management, refine pricing strategies, and elevate overall customer satisfaction.





Problems With Raw Data

Poor data management doesn't let us do successful business because of Limited insights into the market. There are not any Inconsistent pricing strategies and Inadequate tracking of shipping types. Difficulty in understanding customer behavior, resulting in missed opportunities for targeted marketing.



Benefits of Implementing a Data Warehouse



Better Insights for Market



Optimized Cost Tracking



Customer Segmentation &
Revenue Trend Analysis



Enhanced Decision-Making & Business
Performance



Improved Strategy for Local &
International Markets



Data Is Just Everything

The Local Sales (USA) dataset covers products, sales, customers, locations, branches, and suppliers within the U.S. The International Sales (Global) dataset includes the same details for international trades, with additional shipping information.



It is about
the **Customer**



It is about
their behavior, **Sales**,
Revenue and **Profit**



It is about
their wants, **Products**
and their **Categories**



It is about
Location: City,
State/Country



It is useful
in many fields, like
for **Branches**,
Suppliers or
Shippings

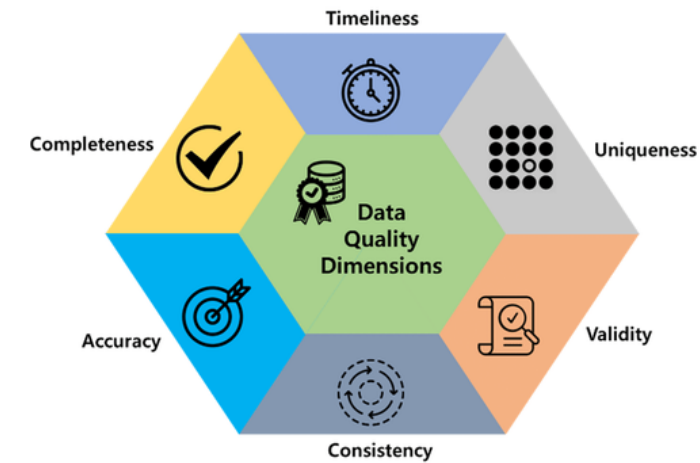
Business Process



Define Grain



Dimensions



Facts

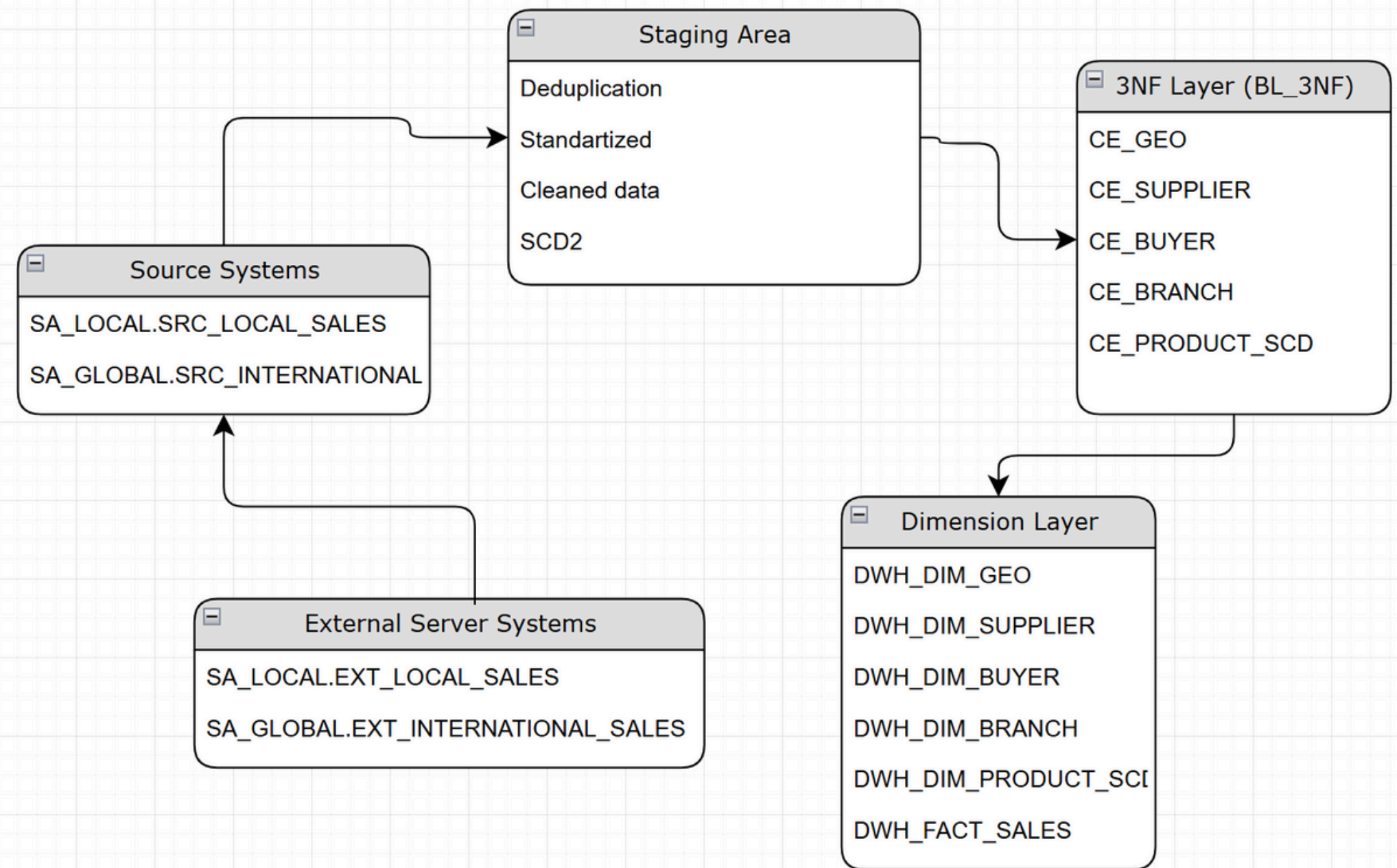


4-Step Process for Grain of DATA

- **Business Process:** Sales & profitability tracking across regions
- **Grain:** Each row = One product sale (Order_ID + Product_ID)
- **Dimensions:** Time, Product, Buyer, Branch, Location
- **Facts:** Sales Price, Revenue, Product Cost, Profit

Creation Flow of DWH

A Data Warehouse (DWH) is designed to efficiently integrate, cleanse, and structure data for reliable analytics and reporting. The process begins with raw data ingestion from multiple external and source systems, ensuring minimal load on core storage while maintaining scalability. The staging area serves as an intermediary layer where data is standardized, deduplicated and transformed, enabling historical tracking through Slowly Changing Dimensions (SCD2). The 3NF layer ensures data normalization, reducing redundancy and preserving relational integrity. Finally, the dimensional layer restructures the data into a star schema, optimizing it for fast query performance, aggregations, and business intelligence applications. This structured approach enhances data accuracy, consistency, and accessibility, ensuring a high-performance analytical environment.



01 Data Ingestion - External (virtual) and Source Tables (Physical)

02 Staging & Transformation - Cleans, standardizes, and prepares data for structuring

03 3NF Layer - Ensures normalized, well-integrated data for consistency.

04 Dimension Layer - Supports fast reporting and business intelligence insights

External and Source Tables

01 Foreign Data Wrappers (FDW) for CSV Handling

- **Why?** Enables reading external CSV files as database tables, avoiding direct imports.
- **Separate Servers** for local_sales and global_sales
- **Foreign Tables** (ext_local_sales, ext_international_sales) to reference external CSVs.

02 Schema-Based Data Segmentation

- **Why are tables in different schemas?**
- **Modularity:** Separates local and international data for better management.
- **Security & Performance:** Limits access and optimizes query execution.
- **Scalability:** Supports additional sources without altering existing structures.

03 Source Table Creation (Structured Storage)

- **Why?** Ensures clean and structured raw data storage before transformation.
- **Implementation:**
 - src_local_sales → Stores local sales data
 - src_international_sales → Stores international sales data
- **Primary Keys:** Ensures uniqueness (order_src_id)
- **Timestamps:** insert_dt for tracking data loading times.

04 Incremental Data Loading (ETL Process)

- **Automated & Efficient Loading**
 - Extract new data from external sources without duplicates
 - Log every operation for restartability
- **Key Procedures:**
 - incremental_load_local_sales()
 - incremental_load_international_sales()
 - master_incremental_load() (Triggers both loads)
- **Prevents duplicate records**
- **Automatically updates last successful order date**
- **Error Handling & Logging**

Final Thoughts

- **Schema-based approach** ensures modularity, scalability, and security.
- Foreign Data Wrappers enable seamless **external data handling**.

Cleaning Layer (CL)

Data Processing

01 Data Cleansing & Standardization

- **Why?** Prevents incomplete or inconsistent records from moving forward.
- **Duplicate handling** ensures unique records using **ON CONFLICT**.
- **Null handling** replaces missing fields with **default values** ('N/A', 0, 0.0).
- **Data type normalization** enforces strict column types for stability.

02 Single Schema with Separate CL Tables

- **Why store all CL tables in one schema?**
 - **Simplicity:** Easier management with a single namespace.
 - **Performance:** Avoids unnecessary cross-schema joins.
 - **Scalability:** New tables integrate seamlessly.

03 Source Table Creation (Structured CL Storage)

- **Why?** Ensures consistent default values for missing data.
- **Implementation:**
 - **ORDER_DT** defaults to the current date if missing.
 - Text fields (**LOCATION**, **CITY**, etc.) default to 'N/A' to prevent NULL issues.
 - Numeric fields (**SALES_PRICE**, **SUPPLIER_RATING**, etc.) default to 0 for stability.
 - Primary keys enforce uniqueness (**ORDER_SRC_ID**).

04 Incremental Data Processing (SRC into CL)

- **Automated and Efficient Transformation**
 - Moves only new records based on **ORDER_DT**.
 - Logs every step for auditability and rollback.
 -
- **Benefits:**
 - Prevents incorrect or missing data from propagating further
 - Ensures data consistency for the next DWH layers
 - Implements error handling and logging for stability

Final Thoughts

- **Default values** prevent missing data from breaking queries.
- **Single schema** design simplifies processing and maintenance.
- **Duplicate handling** ensures unique records using **ON CONFLICT**.

Third Normal Form (3NF) Data Processing

01 SCD2 Implementation for Historical Tracking

- **Why?** Maintains a history of changes for slowly changing dimensions.
 - **Start and End Dates:** Defines the active period for each record.
 - **Is Active Flag:** Identifies the latest version ('Y' for active, 'N' for historical).
 - **Foreign Key Consistency:** Ensures references to the correct product version over time.
- Preserves historical changes for accurate trend analysis.
- Supports time-based queries without data duplication.
- Ensures seamless integration with fact tables.

02 Partitioning for Performance

- **Why?** Optimizes query execution and storage efficiency.
 - Fact table partitioned by order_dt for faster lookups.
 - Quarterly partitions created dynamically for scalability.
 - Efficient data retrieval by limiting scans to relevant partitions.

03 Materialized Views for Query Optimization

- **Why?** Enhances reporting speed by reducing the need for full table scans.
 - Maintains a rolling one-day snapshot for recent transactions.
 - Indexed access improves query performance for dashboards.

04 Source Triplet for Data Lineage

- **Identifies Data Origin:** Shows where each record comes from.
- **Prevents Duplicates:** Ensures data is not repeated or wrong.
- **Helps with Auditing:** Makes it easy to check and fix data issues.
- **Supports Multiple Sources:** Combines data from different places correctly.

Final Thoughts

- **SCD2** ensures accurate historical tracking.
- **Partitioning** enhances query efficiency and scalability.
- **Materialized views** reduce computation time for analytics.
- **Automation minimizes** manual intervention and maintenance overhead.

Dimensional Model (DM) - Final Layer

01 Optimized for Reporting

- Restructures data to **simplify queries and improve performance**.
- **Organizes facts and dimensions** for efficient aggregations.

02 Fact Table Design

- Stores transactional data with foreign keys to dimension tables.
- Includes pre-aggregated metrics like revenue, profit, and cost.
- Partitioned by order_dt for performance.

03 Incremental Data Loading

- Loads new transactions only, avoiding reprocessing old data.
- Uses source triplet to track origin and ensure data consistency.

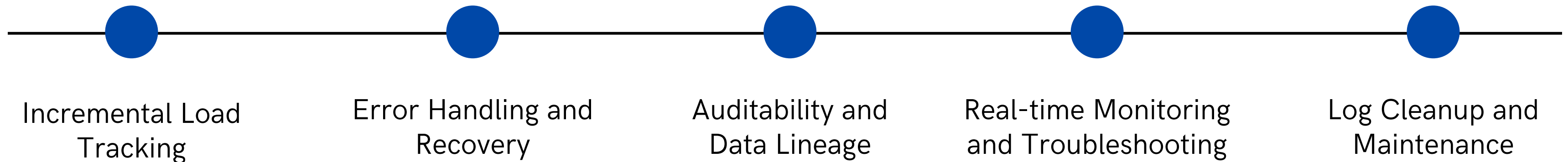
Final Thoughts

The DM layer plays a critical role in transforming raw transactional data into a structured format that is optimized for analysis, decision-making, and reporting. Unlike raw or normalized data, which may require multiple joins and transformations to extract meaningful insights, the DM structure ensures that data is pre-organized, pre-aggregated, and efficiently stored for analytical workloads.

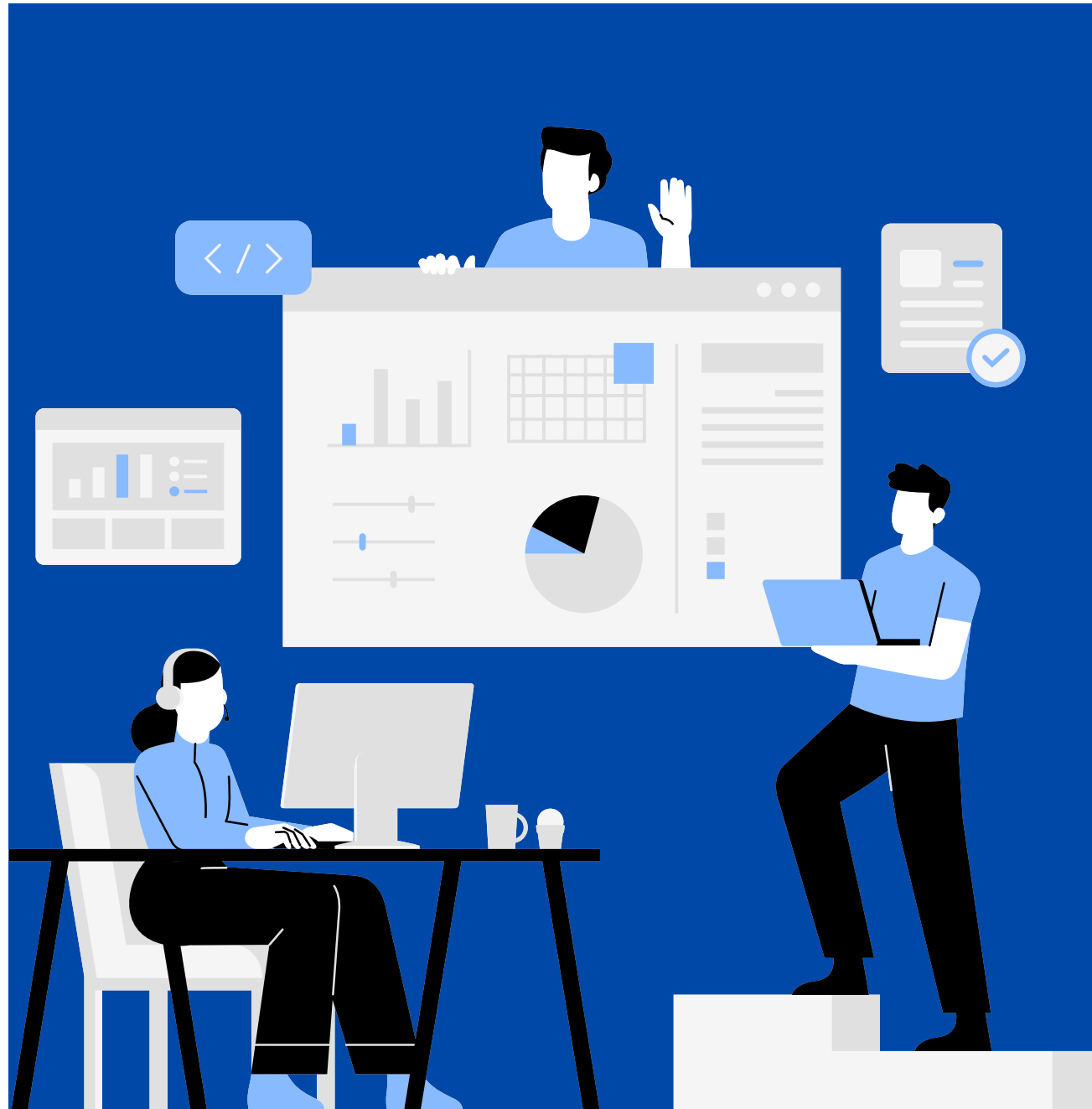
Ultimately, the DM layer transforms complex transactional data into a well-structured, high-performance format, allowing businesses to gain insights quickly, accurately, and efficiently while maintaining a system that is scalable and easy to manage.

Logging and Monitoring in DWH

The logging system ensures **data integrity, traceability, and efficient error handling** by tracking the last successfully processed record to prevent duplicates and maintain incremental loads. Error handling captures failures and enables restartability, allowing retry from the exact failure point. Logging execution details like inserted rows, timestamps, and statuses improves auditability and supports debugging. Real-time monitoring helps quickly identify failed loads, reducing downtime and improving ETL reliability. Automated log cleanup prevents table bloat while retaining necessary historical data, ensuring a scalable and efficient logging framework.



Incremental Load Validation in Logging



01

Tracking Last Processed Record

Logs the last successfully processed `order_dt` to ensure new records are loaded without duplication.

02

Detecting and Inserting New Data

Compares incoming data with the last logged record and inserts only those with a later date.

03

Error Handling and Recovery

If a failure occurs, logs the error message and prevents partial or duplicate processing.

04

Monitoring Load Status

Tracks row counts, timestamps, and success/failure status for auditing and debugging.

Incremental Load Process in DWH

Incremental loading ensures that only new records are processed efficiently. The system first retrieves the latest processed timestamp from a materialized view, avoiding full table scans. Fact tables are partitioned by `order_dt`, reducing query time by scanning only relevant partitions. Indexes on foreign keys and date columns speed up joins and prevent duplicate inserts. The process uses `ON CONFLICT` or `MERGE` to insert new records while maintaining data integrity. To keep performance optimal, materialized views are refreshed, and old partitions are archived or dropped, ensuring efficient storage and query execution.



Faster Data Processing



Optimized Query Performance



Efficient Storage Management



Reliable Incremental Loading



Scalable and Maintainable Architecture



Data Quality

Validation Approach

To ensure **data integrity and consistency**, I implemented validation checks focusing on **duplicate detection** and **completeness verification** between layers. The validation strategy involves querying target tables for **unexpected duplicates** and ensuring all records from the SA layer are properly transferred to the business layer (BL/DM).

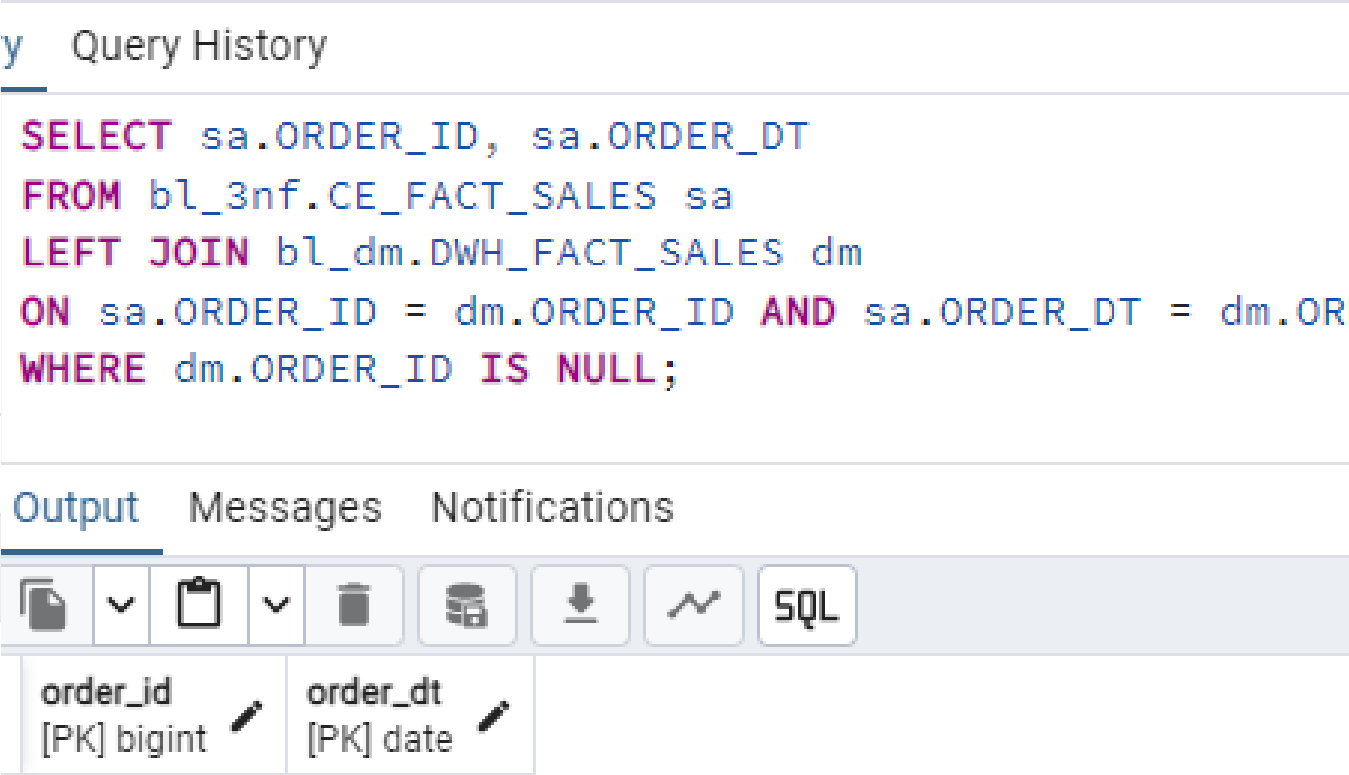
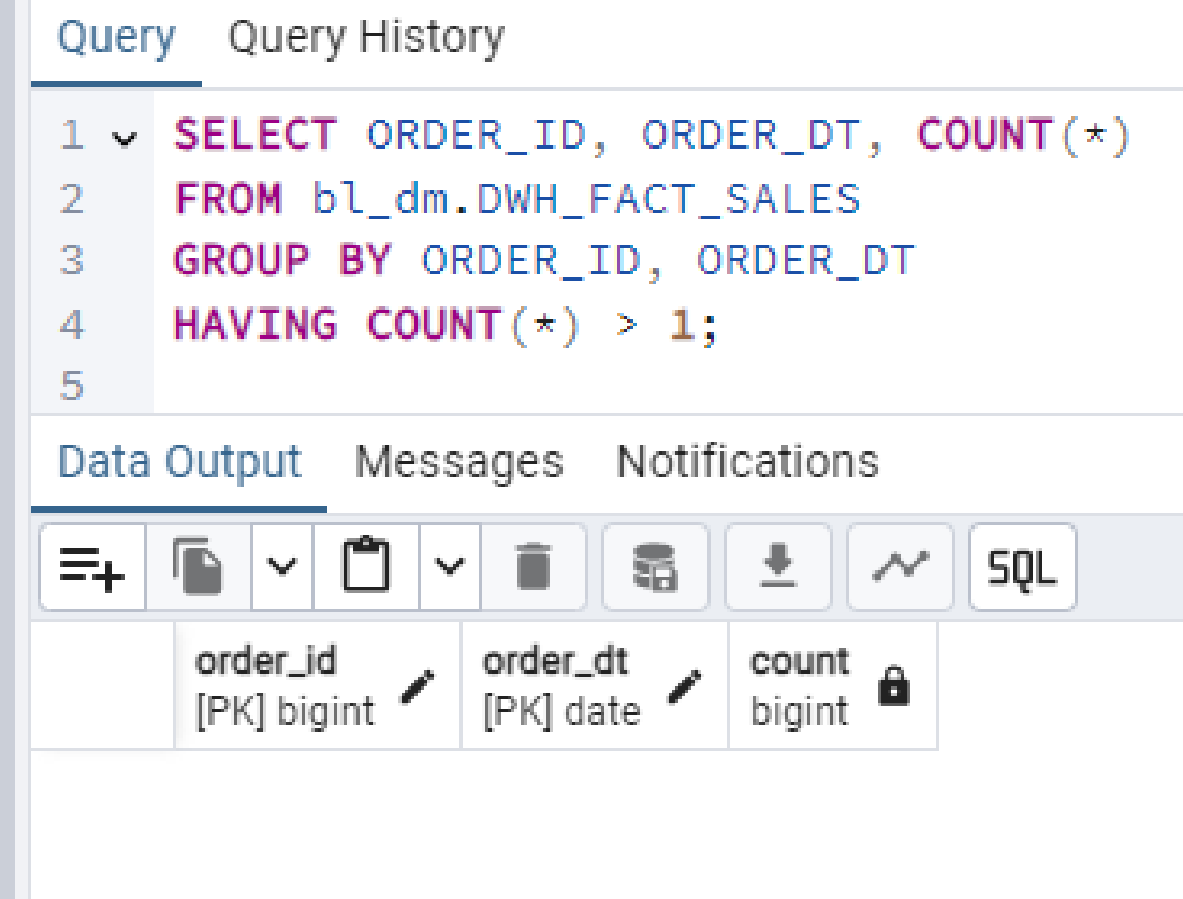
- 01

Duplicate Check - Uses COUNT(*) with GROUP BY to detect duplicate order_id or other key columns.
- 02

Completeness Check - Uses LEFT JOIN or EXCEPT queries to find missing records.
- 03

Referential Integrity Check- Uses NOT EXISTS queries to detect orphan records.
- 04

Data Type and Constraint Validation - Uses CHECK CONSTRAINTS and direct queries to flag incorrect data.



- Expected Outcome:**

- The query should return zero rows if no duplicates exist.
 - If duplicates are found, it indicates an issue with incremental load logic or merging strategies.
- Expected Outcome:**

- The query should return zero rows, meaning all records from SA are properly loaded into BL/DM.
 - If records are missing, it could mean:
 - Filtering conditions removed valid records.
 - ETL failures skipped some records.
 - Load dependencies were not executed in the correct sequence.

Automated Partition Management in DWH

Dynamic Partition Creation

The procedure automatically creates missing partitions (dwh_fact_sales_YYYY_MM), preventing manual updates.

Performance Optimization

Queries scan only relevant partitions, avoiding full-table scans and improving speed.

Duplicate Prevention

Uses SELECT EXISTS to avoid creating partitions that already exist, ensuring data integrity.

Scalability

Handles future partitions up to 2026 and can be extended without modifying the logic.

Cursor & EXECUTE: A cursor iterates over date ranges to generate partition names dynamically, ensuring each partition is created only if missing. EXECUTE runs dynamic SQL statements, allowing the procedure to check for existing partitions and create new ones on demand.

	tablename name
11	dwh_fact_sales_2022_10
12	dwh_fact_sales_2023_01
13	dwh_fact_sales_2023_04
14	dwh_fact_sales_2023_07
15	dwh_fact_sales_2023_10
16	dwh_fact_sales_2024_01
17	dwh_fact_sales_2024_04
18	dwh_fact_sales_2024_07
19	dwh_fact_sales_2024_10
20	dwh_fact_sales_2025_01
21	dwh_fact_sales_2025_04
22	dwh_fact_sales_2025_07
23	dwh_fact_sales_2025_10
24	dwh_fact_sales_2026_01



The background is a blurred image of a data dashboard with a blue color scheme. It features several charts: a donut chart at the top left with the value '16,151', a bar chart at the top right, a line chart in the middle right, and another donut chart at the bottom left with the value '3,739,958,276'. There are also various tables and smaller charts visible in the background.

Thanks

Beka Chokuri